

CLOUD FILE SYNC BOT

A PROJECT REPORT

Submitted by

TAMANNA(23BCS11010)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING



Chandigarh University

JULY, 2025



BONAFIDE CERTIFICATE

Certified that this project report “**CLOUD FILE SYNC BOT**” is the bonafide work of “**Tamanna(23BCS11010)**” who carried out the project work under my/our supervision.

SIGNATURE

Dr. Sandeep Singh Kang

ASSOCIATE DIRECTOR (CSE-3rd YEAR)

SIGNATURE

Er. Roop Sharma

SUPERVISOR

Submitted for the project viva-voce examination held on.

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

I would like to express my heartfelt gratitude to everyone who supported and guided me throughout the development of this project, “**Cloud File Sync Bot**”

First and foremost, I extend my sincere thanks to my project guide for their constant encouragement, timely feedback, and valuable suggestions, which were instrumental in shaping this project effectively.

I am deeply thankful to our institution and the faculty members of the **Department of Computer Science** for providing the necessary resources and a supportive environment for learning and growth.

I also appreciate my friends and classmates for their continuous motivation and for engaging in discussions that contributed valuable insights.

Lastly, I am grateful to my family for their unwavering support, patience, and understanding during this journey.

This project has been a significant learning experience, and I genuinely appreciate everyone who contributed to its success.

Tamanna

23BCS11010

TABLE OF CONTENTS

CHAPTER 1. INTRODUCTION.....	4-7
1.1. Introduction to Project.....	4
1.2. Identification of Problem	5
1.3. Identification of Tasks.....	6
1.4. Timeline.....	7
1.5. Organization of the Report.....	7
CHAPTER 2. BACKGROUND STUDY.....	8-12
2.1. Timeline of the reported problem	8
2.2. Proposed solutions.....	9
2.3. Bibliometric analysis	10
2.4. Review Summary	11
2.5. Problem Definition	11
2.6. Goals/Objectives	12
CHAPTER 3. DESIGN FLOW/PROCESS	13-19
3.1. Evaluation & Selection of Specifications/Features.....	13
3.2. Design Constraints	14
3.3. Analysis and features finalization subject to constraints	15
3.4. Design Flow	15
3.5. Design Selection	17
3.6. Implementation Plan	19
CHAPTER 4. RESULTS ANALYSIS AND VALIDATION	20-21
4.1. Implementation of solution.....	20-21
CHAPTER 5. CONCLUSION AND FUTURE WORK.....	22-23
5.1. Conclusion	22
5.2. Future work.....	23

List of Figures

Figure 1 Timeline.....	7
Figure 2 System Architecture Diagram	14
Figure 3 Block Diagram.....	19
Figure 4 App Main Menu	29
Figure 5 Dashboard.....	29
Figure 6 File Monitoring.....	30
Figure 7 Automatic Drive Upload.....	30

List of Tables

Table 1 Task Breakdown	6
Table 2 Report Organization.....	7
Table 3 Bibliometric analysis	10
Table 4 Goals & Objectives	12
Table 5 Constrains	14
Table 6 Features	15
Table 7 Alternative designs.....	16
Table 8 Required Tools.....	26

ABSTRACT

With data in a dynamic state and collaboration ongoing in today's digital world , having the capability to track and synchronize files between systems in real time is crucial. “**The Cloud File Sync Bot**” project offers an easy-to-install, easily customizable solution that allows users to monitor file system actions—like creation, modification, and deletion—on a single or across multiple folders on a local system. This system is constructed with a solid backend in C# and ASP.NET Core, taking advantage of the strength of FileSystemWatcher to immediately catch changes. Those events are logged and presented to a tidy, responsive frontend interface constructed with HTML, CSS, and JavaScript, enabling users to see logs in real-time, filter by folder, search certain file events, and export logs as CSV files for analysis or auditing purpose sit. It includes cloud integration support, allowing automatic upload of newly created or modified files to Google Drive for backup and accessibility. This extends its utility beyond simple monitoring to partial cloud synchronization as well is geared towards awareness of file activity, which makes it perfect for version tracking, local network monitoring, or research-based use-cases. With its support of viewing log details, folder-level filtering, and clean logging infrastructure, the project embodies the main values of contemporary software engineering—user-oriented design, real-time responsiveness, and modularity. This report details the entire development life cycle of the Cloud File Sync Bot—from problem analysis and design plan to implementation, testing, and future extensions—underlining its value as an effective utility for intelligent file monitoring and system openness.

CHAPTER-1

INTRODUCTION

1.1 Introduction to project

As office spaces rapidly get digitized and the trend of cloud services gains momentum, the need for real-time monitoring and synchronizing files has exponentially grown. Businesses and individuals both need data storage solutions that offer insights into activity at the file level—what was modified, when, and by whom. Such visibility is particularly important in shared environments where many users touch a common set of data. The Cloud File Sync Bot project is designed keeping this need in mind. It is a lean, self-contained system that is intended to track file changes in a directory or directories and report these events in a systematized and user-friendly manner. While the initial focus was limited to local file monitoring, the system now includes Google Drive integration for real-time upload of file changes to the cloud — offering users both visibility and redundancy. In addition it concentrates on the event detection, logging, and visualization of file system activity, giving a precise history of file operations like creation, modification, and deletion.

The application's backend is done in C# and ASP.NET Core, utilizing the FileSystemWatcher class to track directories with minimal system resource utilization. Triggered events are redirected to the frontend using RESTful APIs. The frontend, built with HTML, CSS, and JavaScript, updates the events in real-time on an interactive dashboard. It has facilities such as folder filtering, searching, exporting logs, and viewing events in detail—all accessed through a web browser. It is especially beneficial in academic laboratory, small business, and development environments where it is vital to know local file activity. It fills the gap between pure monitoring and complete cloud solutions and presents an efficient and adjustable middle-way. Not only does the Cloud File Sync Bot solve a pertinent technical issue, but it is also an exercise in applying fundamental software engineering principles of modular design, event-driven programming, and real-time user interface integration.

1.2 Introduction of problem

With today's digitalized spaces, file change monitoring and control have become vital, particularly in collaborative environments, learning labs, and development environments. Users tend to engage with key files that are constantly updated, moved, or deleted, and missing any of these can lead to data loss, project failure, or diminished accountability. Although cloud-based offerings such as Google Drive or Dropbox are available, these offerings are geared mostly toward synchronization and storage functions and not necessarily giving end-user real-time insight into the changes that are happening locally.

The core problem is that there are no lightweight and easy-to-use tools that provide on- going monitoring of a single or more local directories. Most existing solutions are embedded into big business systems or too complicated for non-technical users. Moreover, a lot of tools do not provide such features as filtering out changes, exporting logs, or a customizable interface that adjusts to user requirements.

This project seeks to address that gap with the creation of the Cloud File Sync Bot—a clever, real-time folder tracking system that enables users to monitor file events such as file creation, deletion, and modification. It aims for simplicity, transparency, and instant feedback in a user-friendly web-based dashboard, enabling users to take greater control and knowledge of file operations.

1.3 Identification of Tasks

To address the problem, the project was broken in following phases:

Task Breakdown:

Table 1 Task Breakdown

S.no	Task	Description
1	Problem Identification	Understanding the need for real-time file change tracking in local directories.
2	Requirements Gathering	Listing out both functional and non-functional requirements for the system.
3	Designing Architecture	Planning the overall structure using a client-server model with API integration.
4	Backend Development	Implementing the monitoring logic using FileSystemWatcher in C#.
5	Frontend Design	Creating a dashboard with HTML, CSS, and JavaScript for log visualization.
6	API Integration	Connecting frontend and backend using ASP.NET Core Web APIs.
7	Feature Implementation	Adding search, folder filters, export logs, and clear logs functionalities.
8	Testing and Debugging	Performing functional testing and bug fixing.
9	Report Writing and Documenta-tion	Documenting the technical aspects and writing the final report.

1.4 Timeline

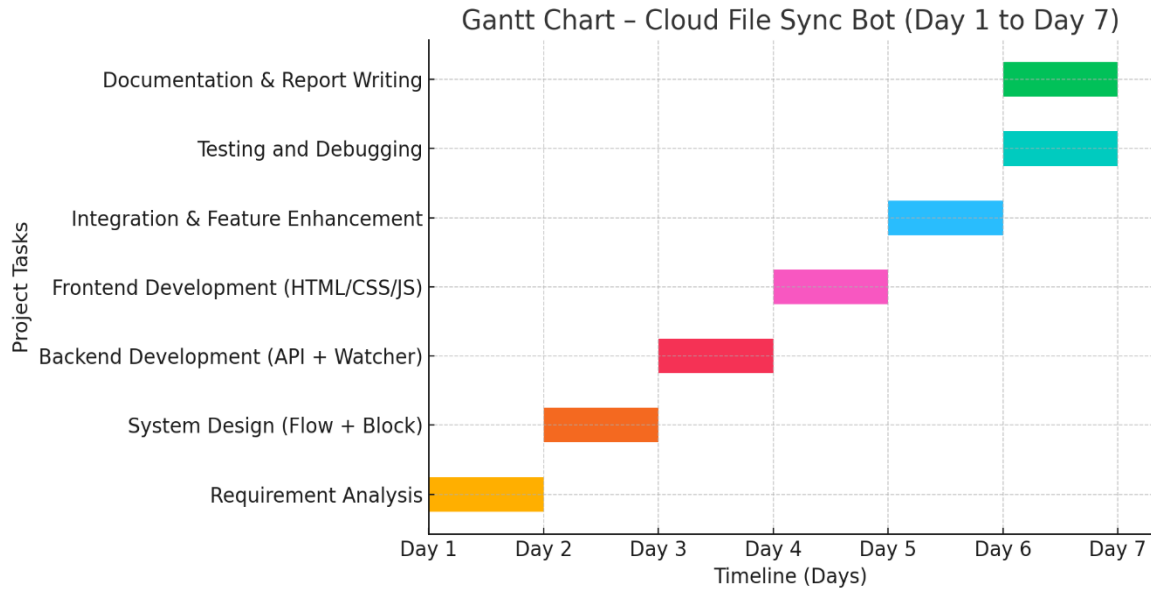


Figure1 TimeLine

1.5 Organization of the report

Table 2 Report Organization

Chapter	Title	Description
Chapter 1	Introduction	Explains the motivation, objective, and importance of developing a file monitoring system using C#.
Chapter 2	Literature Review	Reviews existing file monitoring tools, system watcher concepts, and supporting .NET technologies.
Chapter 3	System Design	Describes the system architecture, folder watcher mechanism, and frontend-backend flow.
Chapter 4	Implementation	Discusses backend logic, frontend integration, API creation, and logging feature development.
Chapter 5	Testing and Results	Showcases screenshots, tests log tracking accuracy, and discusses UI responsiveness.
Chapter 6	Conclusion	Summarizes key outcomes, limitations, and possible improvements such as cross-platform syncing.
Appendix	Code Snippets and Screenshots	Contains code examples, configuration details, and references.

CHAPTER-2

BACKGROUND STUDY

2.1 Timeline of the Reported Problem

The online presence of accounts and services, as well as the number of them, increased sharply. As digital data and cloud computing took center stage, there has been a growing necessity to track file activity both in individual and organizational processes. At first, users did not have easy ways to monitor what changes were happening within their local folders, particularly in real time. The transparency issue resulted in issues like unintended file deletions, overwrites without detection, and a lack of knowing who touched a file.

Prior to cloud services such as Google Drive and OneDrive, file versioning as well as activity tracking were cumbersome and very error-susceptible. Though the cloud platforms brought about automatic synchronization, they never provided a detailed history of file activities for the end users, especially for offline or localized environments. Additionally, most users especially students, researchers, as well as developers—do not always need full cloud synchronization, but they need to know what changed within certain folders during the course of work.

As time passed, developers started investigating using OS-level file watchers like the `FileSystemWatcher` class in .NET to create specialized monitoring tools. But these were usually command line or had clumsy interfaces. They also usually involved single-folder monitoring without live dashboards, search capabilities, or filters.

Cloud File Sync has been designed to fill this void—through a straightforward, multi-folder, GUI-based file tracking system, suitable for desktop use where real-time awareness and cloud backup, are essential with minimal third-party dependencies.

According to a report by IDC, over 60% of organizations experience data loss due to human error or unsynchronized systems. Moreover, Statista (2023) highlights that cloud-based backup adoption has grown by 300% in the last five years, emphasizing the need for automated file sync solutions.

2.2 Proposed Solutions

Several solutions have come up over the years to help solve the issue of file monitoring and synchronization, particularly in business and collaborative settings. These are primarily cloud-based platforms, command-line utilities, or native operating system tools. Yet, they all have some limitations when it comes to real-time local monitoring with an easy- to-use interface.

1. Cloud Storage Platforms (e.g., Google Drive, OneDrive, Dropbox)

- These applications offer instant file synchronization and some rudimentary file versioning. They:
Need internet connection
- Don't present extensive logs of changes (e.g., timestamps, change type) Are not designed for local-only folder activity monitoring
- Even though they address the backup and availability issue, they are less suited for users who need to monitor file events without uploading data to the cloud.

2. OS-Level Utilities (e.g., Windows Event Viewer, Audit Policy)

These are native facilities that can trace some file system actions.

But they:

- Are hard to set up.
- Need administrator access.
- Do not have user-friendly interfaces for non-professionals.

They are good for system administrators but are not aimed at day-to-day folder watching by ordinary people.

3. Third-Party File Watcher Tools

Some tools are available which are open-source or paid and they offer simple file watching functionality.

Most of them:

- Only monitor one folder at a time.
- Do not support real-time dashboard visualizations.
- Have limited support for exporting logs or filtering results

4. Code-Based Solutions (e.g., PowerShell Scripts, Python Watchdog, .NET FileSystemWatcher)

Most developers develop small-scale scripts using these libraries. For instance:

- PowerShell's Register-ObjectEvent.
- Python's watchdog module
- .NET's FileSystemWatcher class

2.3 Bibliometric Analysis

Table 3 Bibliometric analysis

Tool / Method	Key Features	Effectiveness	Drawbacks
FileSystemWatcher (.NET)	Monitors file creation, modification, and deletion in real-time	Offers native integration, low memory usage, instant detection	Limited to local folders and Windows platform only
Google Drive API	Uploads files to Google Drive automatically	Ensures backup & accessibility	Needs credentials.json & user authorization
ASP.NET Core Web API	RESTful backend for handling folder inputs and log access	Clean modular architecture, supports real-time interaction	Requires app to be actively hosted during use
In-Memory Log Manager	Stores logs temporarily without database dependency	Fast access, easy to reset, no DB setup required	Logs are not persistent — lost on application restart
JavaScript Fetch API	Periodically fetches logs from backend to update dashboard	Enables live updates on UI without reloading	Not true real-time; depends on polling interval
CSV Export Function	Allows users to export log history for backup/reporting	Easy for users to download logs for later use	No automated backup or cloud integration
Manual Folder Input	User-specified monitoring folders via frontend UI	Flexible and allows tracking multiple directories	No drag-and-drop or folder tree view interface
HTML/CSS Dashboard UI	Responsive table with filtering, searching, clearing options	Intuitive and clean for end-user experience	Lacks login/authentication; not secure for multi-user scenarios

2.4 Review Summary

From the analysis of available tools, libraries, and technologies, it becomes clear that although several options are available for file backup as well as synchronization, there is a large void in local file change visibility and real-time detection.

Cloud solutions such as OneDrive and Google Drive provide synchronization but are not log-based transparent. Command-line scripts and tools, while capable, are impractical for the non-technical person. It is also not supported by multi- folder monitoring, customizable dashboards, and functions such as exporting logs, search, or filter activity logs within an easy interface.

This made us realize the imperative for a lightweight, GUI-oriented system that could be run locally, operate independently of the cloud, and provide end-to- end visibility into file system changes in a easily understandable format.

To remedy these shortcomings, the Cloud File Sync Bot was created, sporting a clean, utilitarian interface atop a reliable C# monitoring backend.

2.5 Problem Definition

The issue this project addresses is the absence of a light, multi-folder file activity tracking system that is user-friendly to the average user and operates in real time. Users frequently experience instances where files have been accidentally erased, overwritten, or altered—without a specific record of when or how these events have transpired.

There is no Windows utility that provides folder-level change logging with filtering, search, and export. Current third-party utilities are too technical, too constrained, or too part of other platforms that do not address this exact requirement.

Therefore, the aim is to create a system that:

- Tracks user-configured folders in real-time.
- Notices file creation, change, and deletion events.
- Provides real-time logs on a web-based dashboard.

- Offers necessary utilities such as folder filtering, search, and export.
- Does not need any technical setup and does not require much system resources.
- To enable automatic cloud-based backups using Google Drive, ensuring real-time file protection and remote accessibility.

The Cloud File Sync Bot provides this specialized solution with technologies such as C#, ASP.NET Core, and JavaScript, providing both backend speed and frontend user-friendly experience.

2.6Goals/Objectives

Table 4 Goals & Objectives

S. No.	Objective	Description
1	Real-time folder monitoring	Detect file creations, deletions, and modifications instantly.
2	Cloud sync integration	Upload modified or new files to Google Drive.
3	Multi-folder support	Allow monitoring of more than one folder at the same time.
4	Interactive dashboard	Design a responsive web interface to display file change logs.
5	Search and filter functionality	Enable users to search specific entries and filter logs by folder.
6	Export logs to file	Provide option to download logs in CSV format for backup or reporting.
7	Clear log history	Add feature to reset or clear displayed logs with a single click.
8	Seamless backend and frontend integration	Ensure smooth data flow using ASP.NET Core APIs between UI and backend logic.
9	Minimal system resource usage	Ensure the application runs in the background without affecting system speed.

CHAPTER-3

DESIGN FLOW/PROCESS

3.1 Evaluation & Selection of Features

Before initiating development of the Cloud File Sync Bot, several feature options were considered. The objective was to create something that is minimalist, yet functional and realistic for individuals looking to monitor changes to local folders without entirely depending on the cloud and intricate installations.

Following examination of several current tools and analyzing the needs of users, a list of possible features was generated. The most practical and effective features from this list were chosen based on usefulness, influence, technical viability, and user experience.

Some of the initially considered features, like multi-device bidirectional synchronization and email notifications were not included to preserve the system's simplicity. However, cloud storage integration using Google Drive was implemented to support lightweight, real-time file backup without compromising performance.

The final feature list was tested on the following grounds:

- Relevance to the problem (Is it required to resolve the file visibility problem?)
- Ease of implementation (Can it be developed in the specified time frame?)
- User experience value (Does it enhance how the user uses the system?)
- Only the features that met these specifications were chosen for the finished product.

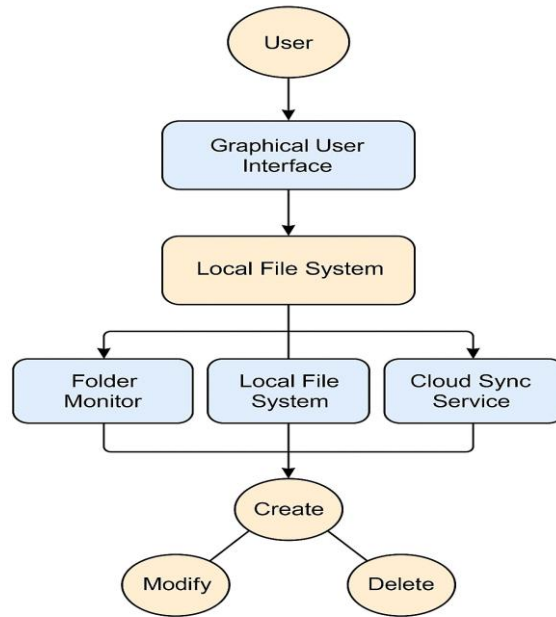


Figure 2 System Architecture Diagram

3.2 Design Constraints

Table 5 Constrains

S. No.	Constraint	Description
1	Local System Dependency	The system is built to run only on Windows OS using FileSystemWatcher (not cross-platform).
2	Cloud API Dependency	The system depends on a stable internet connection and Google Drive API credentials. Any changes to API limits or authentication flow may impact functionality.
3	Single User System	Designed for use on a single user machine, no multi-user account handling.
4	Folder Input Validation	Only valid and accessible local folders can be monitored; network folders excluded.
5	System Resource Efficiency	Monitoring must run in the background without consuming high CPU or RAM.
6	Security Constraints	No direct file access/editing is allowed by the system—only event logging is done.

3.3 Features Finalization

Table 6 Features

S. No.	Feature	Purpose
1	Folder Monitoring Input	To allow users to specify folders they want to track.
2	Real-Time Log Generation	Detect file creation, modification, and deletion events instantly.
3	Cloud Integration	Enables automatic syncing of changed files to Google Drive using Drive API and OAuth authentication
4	Multi-Folder Support	Users can add more than one folder for monitoring.
5	Log Dashboard with Search	View all logs in a table with live search and folder filter.
6	Export Logs (CSV)	Save current logs for backup or analysis.
7	Clear Logs Button	Reset the log view when needed.
8	Live Frontend-Backend Integration	Logs are fetched and displayed automatically using ASP.NET Core API.

3.4 Design flow

The Cloud File Sync Bot design flow specifies how the system works step-by-step, from when the user opens the application to when file changes are detected and reflected in the dashboard.

The process starts by accepting one or more folder paths to watch. The folder paths are then verified by the backend to check if they actually exist and can be accessed. Upon verification, the system enables the FileWatcherService, which employs the .NET FileSystemWatcher to watch the target folders for any change in form of creation, modification, or deletion of a file.

When detection of an event is made, a log entry is generated as a structured one with the file path, change type, timestamp, and source folder. This log is posted to the backend controller via an API call, where it is stored in memory.

Concurrently, the frontend dashboard retrieves current logs via AJAX calls every few seconds and

refreshes the table view in real time. Users are able to filter logs by folder, search for specific entries, delete the log view, or export logs as a CSV file.

After detecting a file change, the system also invokes a cloud upload operation using Google Drive APIs to ensure the changed file is backed up online.

Table 7 Alternative designs

Design Alternative	Description	Pros	Cons
A. Local File Monitoring Only	In this approach, the system only monitors file changes in local folders and displays logs to the user via a GUI. It avoids cloud interaction and works completely offline.	- Simple to implement - Fast & lightweight - No internet required	- No backup support - Files not recoverable after local deletion - Lacks remote accessibility
B. Cloud-Integrated File Monitoring (Chosen)	This design monitors local folders and also syncs changed files to Google Drive using OAuth 2.0 and Google Drive API, enabling cloud backups.	- Real-time cloud backup - Remote access to synced files - Prevents data loss	- Slightly higher complexity - Requires internet & Google account authentication

This modular workflow guarantees that backend and frontend elements process asynchronously promoting seamless user experience and ongoing background monitoring.

3.5 Design Selection

Selected Design: After evaluating multiple possible approaches for monitoring file activity, a modular and event-driven architecture was selected for the Cloud File Sync Bot. The design prioritizes real-time responsiveness, simplicity of use, and separation of concerns between the backend logic and frontend interface.

Two major design paths were initially considered:

Option 1: Local File Monitoring

- Pros:
 - Easier to build and test
 - Lightweight
- Cons:
 - Not user-friendly
 - Difficult to visualize logs or filter them
 - No real-time UI or interaction

Option 2: Web-based dashboard with backend monitoring and cloud integration

- Pros:
 - Clean, modern interface
 - Real-time log viewing and filtering
 - Easily extendable for future features
- Cons:
 - more complex to integrate backend and frontend

After comparison, **Option 2 was selected** as it aligns best with the goal of creating a usable, professional looking application with a live interface. This design enables continuous monitoring via a background service, while users interact with a dynamic frontend that auto-updates using asynchronous API calls.

Moreover, this architecture ensures that:

- The backend (written in C#) handles all logic related to folder watching and log creation.

- The frontend (HTML/CSS/JavaScript) focuses purely on UI and data presentation.
- APIs serve as a clean bridge between both, making the system loosely coupled and easier to maintain or upgrade.

This design decision provided a balance between performance, usability, and scalability — fulfilling the core objectives of the project.

3.6 Implementation Plan/Methodology

Programmatic implementation The deployment of the Cloud File Sync Bot was mapped out into modular stages to avoid any interruption of development or complicated debugging. The application was segregated into different modules, each handling a particular functionality like monitoring files, drive sync, API exchange, logging data, and frontend presentation.

Development was incremental as follows:

Phase 1: Backend Setup

- New ASP.NET Core Web API project created.
- Used FileSystemWatcher in C# to monitor file events in chosen directories.
- Created the LogManager class to cache log entries temporarily in the memory.
- Exposed RESTful API endpoints to accept folder paths and return logs to the UI.

Phase 2: Frontend Design

- Developed a clean and responsive dashboard with HTML, CSS, and JavaScript.
- Added form input to accept folder paths to be monitored.
- Designed a table interface to render logs retrieved from the backend API.
- Implemented AJAX calls to poll logs every few seconds.

Phase 3: Feature Enhancement

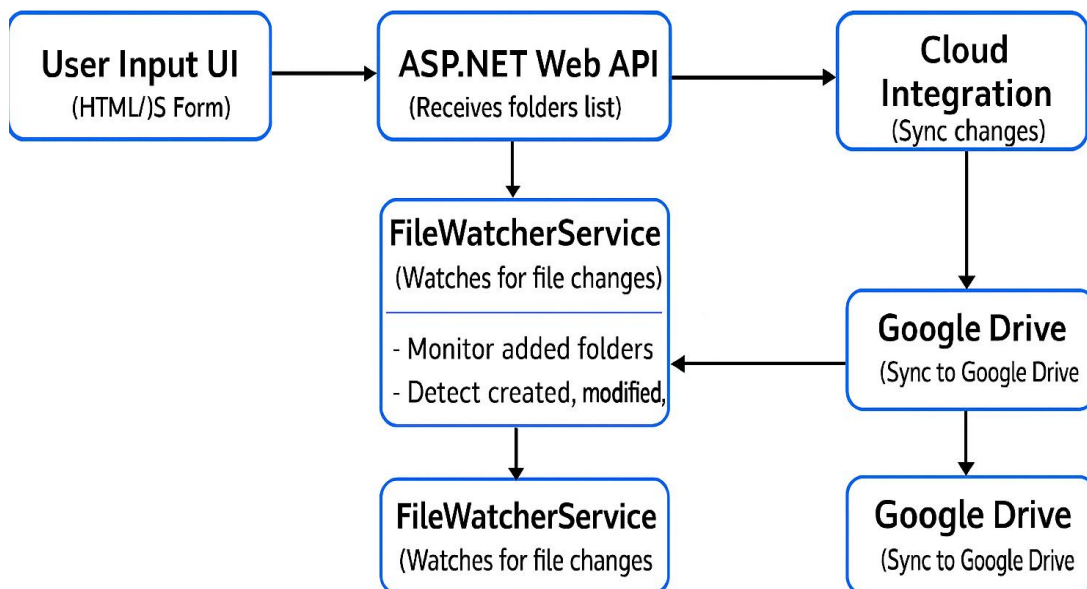
- Included search and filter capabilities within the dashboard.
- Implemented multi-folder support and proper change tracking.
- Added log export (CSV download) and clean logs capability.
- Optimized UI styling and made the dashboard interactive and mobile-supporting.

Phase 4: Cloud Integration and Automation

- Integrated Google Drive API for real-time cloud upload of changed files.
- Implemented secure authentication using credentials.json and OAuth2 flow.
- Added pause and resume functionality to toggle live monitoring dynamically.
- Incorporated upload status tracking, displaying cloud sync status on the dashboard.
- Ensured retry mechanism for locked files and excluded temporary files like ~*.tmp.

Block Diagram:

Figure 3 Block Diagram



CHAPTER-4

RESULTS ANALYSIS AND VALIDATION

4.1 Implementation of Solution

To develop the Cloud File Sync Bot and ensure a smooth and testable system, multiple tools, technologies, and development strategies were used at different stages of implementation. Visual Studio Code was used as the main development environment for writing, debugging, and testing the C# code.

Analysis and Development Tools:

- Visual Studio 2022 was employed as the main IDE for writing the backend logic in C#. It offered functionalities such as IntelliSense, debugging, and API testing support.
- The .NET FileSystemWatcher class was employed to monitor file system changes such as creation, modification, and deletion events in real-time.
- Postman was used from time to time in the development of the backend to test API endpoints and validate data flow prior to integration with the frontend.
- Chrome DevTools assisted in debugging and inspecting JavaScript for live log retrieval and dashboard responsiveness.
- Google.Apis.Drive.v3 – For handling cloud upload via Google Drive.

Design and Architecture:

The architecture followed a modular design approach:

- FileWatcherService.cs – managed folder monitoring logic.
- LogManager.cs – handled temporary in-memory log storage.
- SyncController.cs – exposed REST API endpoints.
- script.js – updated frontend UI by fetching logs from the backend.
- DriveUploader.cs: Service for uploading files to the user's Google Drive via OAuth 2.0 authentication using Google.Apis.Drive.v3

The frontend UI was built using HTML5, CSS3, and vanilla JavaScript, allowing real-time interaction without requiring frontend frameworks.

Report Preparation:

- The project report was prepared using MS Word, following university formatting guidelines.
- Sections were divided based on actual implementation phases (problem analysis, design, development, testing).
- Screenshots of the dashboard, folder input form, log table, and export features were included to demonstrate results.
- A flowchart and block diagram were designed to visually explain system operation.

Project Management and Communication:

- Folder structure in Visual Studio was organized by layers: Controllers, Services, Models, and wwwroot.
- Version control and file backups were maintained locally.
- Manual changelogs were maintained to track feature additions and fixes across development week.

Testing and Validation Tools:

The file monitoring logic was tested by adding, renaming, modifying, and deleting files in test folders. Logs were verified to correctly reflect the type of change, file path, and timestamps.

The UI was validated to ensure that:

- Logs update automatically every 5 seconds.
- Filters and search inputs worked correctly.
- Export ,clear , pause and resume buttons functioned as expected.

CHAPTER 5

CONCLUSION AND FUTURE WORK

5.1 Conclusion

The Cloud File Sync Bot project is able to accomplish its goal of a light-weight, real-time file tracking and logging solution for local folders. It allows users to monitor file system events like creation, change, and deletion with little configuration and high responsiveness.

By merging C#'s FileSystemWatcher with an ASP.NET Core Web API backend in a modular style and an uncluttered HTML/CSS/JavaScript frontend, the program provides a simple-to-use dashboard to see file activity in one or more chosen folders. Search, folder filtering, export of logs, and clear logs features create a user-friendly experience while making the program accessible for academic and personal use.

This version includes a basic cloud integration for uploading newly created files to Google Drive, it offers useful visibility into file changes and assists users in keeping themselves informed of local file activity. The modularity of the implementation also makes it easy to add new functionality in the future, such as adding cloud backup, user authentication, or database-based logging.

Cloud synchronization is partially supported through Google Drive integration, enabling backup of changed files in real-time, making the solution more practical and scalable

In general, the project shows the application of real-time event monitoring and full-stack development with contemporary tools and the clean, scalable architecture that is ideal for academic and light professional environment.

5.2 Future Work

While the current system meets the core requirements, there are several potential areas of improvement and extension:

Full Two-Way Cloud Sync

Expand the current Google Drive integration to cover upload and download capabilities, so both directions of synchronization are supported between cloud and local folders.

Multi-Cloud Platform Support

Support other cloud storage providers such as Dropbox, OneDrive, and AWS S3 and let users select their preferred platform or sync between multiple providers at the same time.

Persistent Log Storage

Have a light local database (e.g., SQLite or LiteDB) save logs to store them for the long term, even across restarts, for long-term activity history and analytics.

Advanced File Filters and Rules

Have users specify file type, size, folder path, or extensions filters — and specify rules to ignore, upload, or mark certain changes.

Scheduling and Automation Options

Allow periodic monitoring sessions, upload periods, and backup snapshots according to user-specified time intervals or system occurrences (such as login or idle duration).

Authentication and Role-Based Access

Implement user login and access restriction to limit who can initiate monitoring, modify cloud settings, or access logs — highly beneficial in shared environments or enterprise environments.

Email or Desktop Notifications

Send real-time notifications for important file changes, upload errors, or sync conflicts through email or system tray notifications.

Cross-Platform Compatibility

Extend compatibility to include Linux and macOS with cross-platform frameworks such as .NET MAUI or Electron, allowing use outside of Windows environments.

Log Visualization and Analytics

Include interactive charts to display file activity patterns, folder-specific change frequency, or user-created events — assisting users in interpreting their system usage.

REFERENCES

1. Chrome Developers – Debugging with Chrome DevTools
<https://developer.chrome.com/docs/devtools/>
2. DotNET Core Tutorials – Building File Watchers in C#
<https://dotnetcoretutorials.com/2020/01/19/using-filessystemwatcher-in-net-core/>
3. FreeCodeCamp – Frontend Integration with REST APIs
<https://www.freecodecamp.org/news/how-to-connect-frontend-to-backend/>
4. GeeksforGeeks – RESTful API Design and Best Practices
<https://www.geeksforgeeks.org/rest-api-best-practices>
5. Microsoft Docs – Create Web APIs with ASP.NET Core
<https://learn.microsoft.com/en-us/aspnet/core/web-api>
6. Microsoft Docs – FileSystemWatcher Class in .NET
<https://learn.microsoft.com/en-us/dotnet/api/system.io.filesystemwatcher>
7. Microsoft Docs – Getting started with ASP.NET Core
<https://learn.microsoft.com/en-us/aspnet/core/introduction-to-aspnet-core>
8. Microsoft Learn – Using In-Memory Collections in ASP.NET Core
<https://learn.microsoft.com/en-us/aspnet/core/fundamentals/in-memory-cache>
9. Mozilla Developer Network (MDN) – JavaScript Fetch API
https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
10. Stack Overflow – Best Practices for File Monitoring using .NET Core
<https://stackoverflow.com/questions/tagged/filesystemwatcher>
11. Visual Studio 2022 Documentation – IDE for .NET Development
<https://learn.microsoft.com/en-us/visualstudio>

APPENDIX

A.1. Project Directory Structure

```
CloudFileSyncBot
└─ Controllers
• SyncController.cs
└─ Services
    └─ DriveUploader.cs
• FileWatcherService.cs
• LogManager.cs
└─ Models
• LogEntry.cs
└─ wwwroot
└─ dashboard
• index.html
• style.css
• script.js
• Program.cs
• appsettings.json
• launchSettings.json
```

A.2. Sample Output

```
Input: Monitoring started for folder C:\Users\taman\Desktop\SyncFolder1
Console Output:
File Created: report.docx at 27-06-2025 11:12:43
File Modified: index.html at 27-06-2025 11:13:10
File Deleted: old_notes.txt at 27-06-2025 11:14:01
```

A.3. Sample Content of appsettings.json:

```
Logging Level: Default – Information
Microsoft.AspNetCore – Warning Allowed Hosts:
```

```
Allow requests from host)
```

A.4. Required Tools and Dependencies

Table 8 required tools

Tool/Package	Purpose
.NET SDK (>= 6.0)	Backend development using C# and ASP.NET Core
FileSystemWatcher (.NET)	Real-time monitoring of folder file system events
Visual Studio 2022	IDE for coding, building, and debugging the solution
HTML/CSS/JavaScript	Frontend dashboard design and live UI functionality
Chrome	Browser to run and test frontend dashboard
Postman (Optional)	API testing during backend development
Google.Apis.Drive.v3	Uploading files to Google Drive via API

A.5. Command Line Steps to Run the Project

1. Open the solution in Visual Studio 2022
2. Make sure .NET 6.0 is installed on the system
3. Set the project startup URL to dashboard/index.html
4. Press **F5** or click **Start Debugging**
5. In the browser, enter folder paths and start monitoring

A.6. Licensing Information

- The project is intended for academic, educational, and personal use only.
- Tools like FileSystemWatcher and .NET libraries are used under Microsoft's official license.
- No third-party packages requiring external licensing are used.
- All frontend assets (HTML/CSS/JS) are custom built for this project.

USER MANUAL

Step 1: Prerequisites

Before you start, make sure the following tools are installed on your system:

- .NET SDK 6.0 or later
- Visual Studio 2022 (or any IDE that supports ASP.NET Core)
- Google Chrome (or any modern browser)
- Windows 10 or higher (FileSystemWatcher is platform-specific)
- Google.Apis.Drive.v3 – For handling cloud upload via Google Drive.

Step 2: Prepare the Project Directory

Download or clone the CloudFileSyncBot project folder. The main structure includes:

- Controllers/ → SyncController.cs
- Services/ → FileWatcherService.cs, LogManager.cs, DriveUploader.cs
- Models/ → LogEntry.cs
- wwwroot/dashboard/ → index.html, style.css, script.js
- Program.cs
- appsettings.json
- launchSettings.json

Make sure these files are not renamed or moved.

Step 3: Set Up Launch Settings

In Visual Studio:

- Right-click the project → **Properties** → **Debug**
- Set the Launch Browser option to: dashboard/index.html
- Make sure the applicationUrl is set to: http://localhost:5018 or your preferred local port
This ensures the dashboard loads first on project start.

Step 4: Build and Run the Project

To run the system:

1. Open the project in Visual Studio
2. Build the solution using Ctrl + Shift + B
3. Start the application using F5 or the **Start Debugging** button
4. The browser will launch at the dashboard page (index.html) If it doesn't, manually go to:
<http://localhost:5018/dashboard/index.html>

Step 5: Using the Application

- Enter a folder path in the input field and click Start Monitoring
- Once started, the system will begin tracking file changes (create, modify, delete) in that folder
- Logs will automatically appear in the dashboard log table in real time
- Modified and newly created files are automatically uploaded to Google Drive, provided the app has been authenticated using credentials.json.
- You can:
 - **Search** logs using the search bar
 - **Filter** logs by folder using the dropdown
 - **Export Logs** as a .csv file for reporting or backup
 - **Clear Logs** using the Clear button to reset the view

Troubleshooting

- If changes are not detected, make sure:
 - The folder path is valid and accessible
 - You have permission to read/write in that folder
 - You are running the app as **Administrator**, if required
- If the browser doesn't open the dashboard, go to:
<http://localhost:5018/dashboard/index.html> manually
- Use Chrome DevTools (F12) to check if logs are being fetched via API
- If logs don't appear, check backend console to verify FileSystemWatcher is active

Images of App

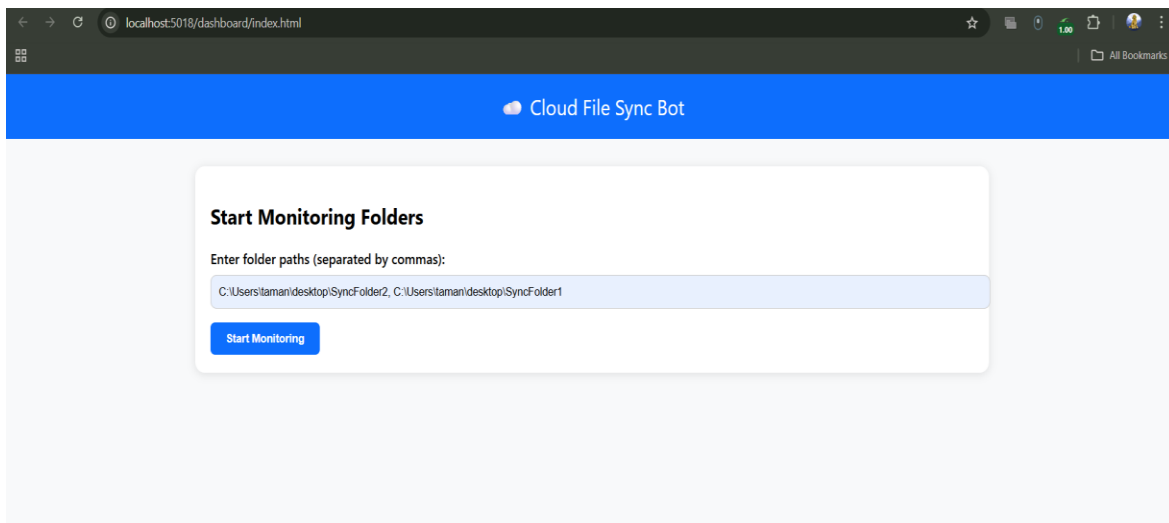


Figure 4 main menu

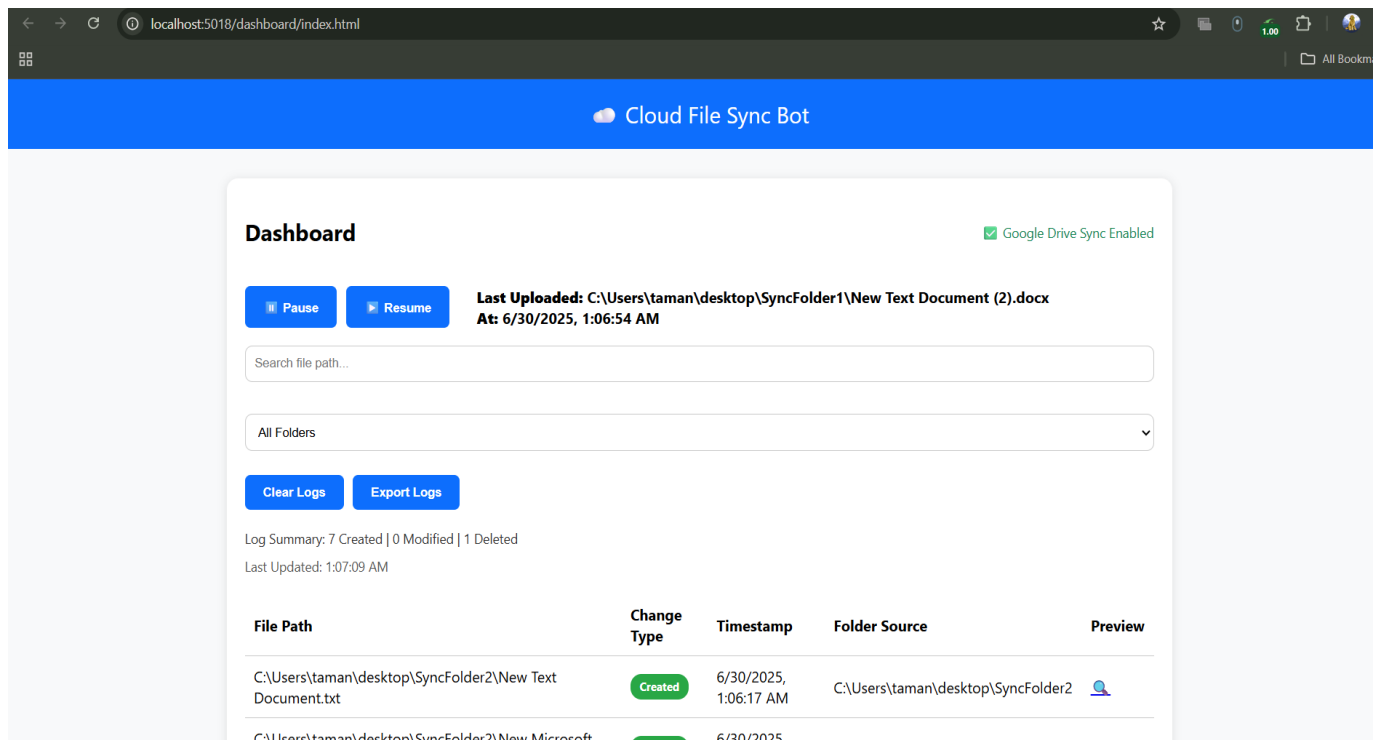


Figure 5 Dashboard

localhost:5018/dashboard/index.html

Log Summary: 7 Created | 0 Modified | 1 Deleted
Last Updated: 1:07:49 AM

File Path	Change Type	Timestamp	Folder Source	Preview
C:\Users\taman\desktop\SyncFolder2\New Text Document.txt	Created	6/30/2025, 1:06:17 AM	C:\Users\taman\desktop\SyncFolder2	
C:\Users\taman\desktop\SyncFolder2\New Microsoft PowerPoint Presentation.pptx	Created	6/30/2025, 1:06:18 AM	C:\Users\taman\desktop\SyncFolder2	—
C:\Users\taman\desktop\SyncFolder2\New Microsoft Word Document.docx	Created	6/30/2025, 1:06:20 AM	C:\Users\taman\desktop\SyncFolder2	—
C:\Users\taman\desktop\SyncFolder2\New Microsoft Word Document (2).docx	Created	6/30/2025, 1:06:25 AM	C:\Users\taman\desktop\SyncFolder2	—
C:\Users\taman\desktop\SyncFolder1\New Text Document (2).txt	Created	6/30/2025, 1:06:33 AM	C:\Users\taman\desktop\SyncFolder1	
C:\Users\taman\desktop\SyncFolder1\New Text Document (2).docx	Created	6/30/2025, 1:06:49 AM	C:\Users\taman\desktop\SyncFolder1	—
C:\Users\taman\desktop\SyncFolder1\New Text Document (2).docx	Deleted	6/30/2025, 1:06:49 AM	C:\Users\taman\desktop\SyncFolder1	—
C:\Users\taman\desktop\SyncFolder1\New Text Document (2).docx	Created	6/30/2025, 1:06:49 AM	C:\Users\taman\desktop\SyncFolder1	—

Figure 6 file monitoring

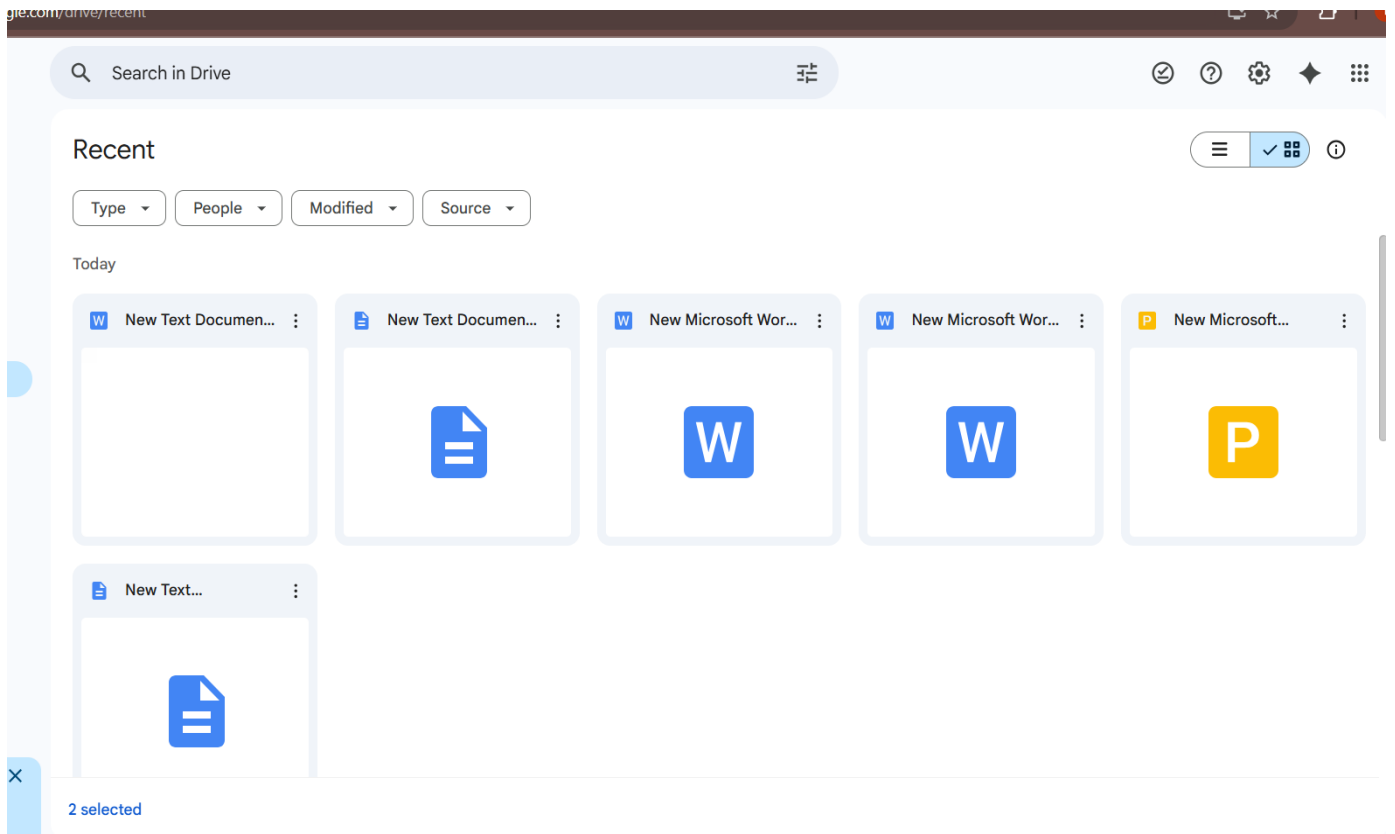


Figure 7 Automatic Drive upload

Licensing

- This project is developed for educational purposes under university guidelines
- No third-party packages requiring license have been used
- All code is custom written or based on open .NET libraries